

Research Notes: 2026 Local RAG API and Data Model Best Practices

1. API Design Trends (July 2026)

- **RESTful Resource Modeling:** Standardizing access to documents, collections, and search results using REST principles [4] [6].
- **Asynchronous Processing:** Handling document ingestion and embedding generation as background tasks with status polling or webhooks [4].
- **OpenAI Compatibility:** Many local RAG systems aim for compatibility with OpenAI's API structure (e.g., `/embeddings`, `/chat/completions` with context) to simplify integration [12].
- **Hybrid Search Endpoints:** Providing unified endpoints that combine vector search and traditional keyword search results [5].

2. Data Model Best Practices (July 2026)

- **Document Schema:**
 - `id` : Unique identifier (UUID).
 - `content` : Raw text or processed content.
 - `metadata` : JSON object containing source, author, date, and custom tags.
 - `embeddings` : Vector representation of the document.
- **Collection Schema:**
 - `id` : Unique identifier.
 - `name` : Human-readable name.
 - `description` : Optional description.
 - `config` : Configuration for the collection (e.g., embedding model, distance metric).
- **Vector Database Standards:** `pgvector` on Postgres is a top choice for production, while ChromaDB and LanceDB are popular for simpler local setups [8] [11].

3. Integration with llama.cpp

- **Context Injection:** The primary method for RAG with `llama.cpp` is retrieving relevant chunks and injecting them into the LLM's prompt context [14] [15].
- **Local Embedding Generation:** Using `llama.cpp`'s embedding models to generate vectors locally for both ingestion and querying [10].

References

1. [REST API for AI Models Explained: 2026 Guide](#)
2. [A practical 2026 roadmap for modern AI search & RAG systems](#)
3. [API design best practices guide \(March 2026\)](#)
4. [Top 15 Vector Databases in 2026: A Production Guide](#)
5. [Is LLM necessary for RAG if we can retrieve answer from vector ...](#)
6. [Best open source vector database solutions: Top 5 in 2026](#)
7. [llama.cpp Tutorial: Run a Local LLM in 12 Steps \[2026\]](#)
8. [Implementing RAG, some questions on llama.cpp #12125](#)
9. [Building a RAG Pipeline with llama.cpp in Python](#)